Spatial and connectivity

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Purpose . . . 1
Source . . . 2
Format . . . 2
Example . . . 4
Related files . . . 4
Purpose . . . 4
Source . . . 5
Format . . . 5
Example . . . 6
Related files . . . 6
Purpose . . . 6
Source . . . 6
hydrology.hyd . . . 6
topography.hyd . . . 8
field.fld . . . 10
shade_factor.shf . . . 12
Related files . . . 14
Purpose . . . 14
Source . . . 14
recall.rec (master database) . . . 14
salt_recall.rec (salt side database) . . . 15
cs_recall.rec (constituent side database) . . . 16
Element and region files . . . 16
Example . . . 16
Related files . . . 17

Purpose

The `.con` files describe the spatial-object topology of the project. Every spatial object (HRU, channel, aquifer, reservoir, routing unit, recall, export coefficient, delivery ratio, outlet, gwflow cell) gets one row in its corresponding `.con` file. The row gives the object its sequential number, name, GIS id, drainage area, location, elevation, parameter pointers, weather station, optional constituent and overbank pointers, and the list of downstream objects that receive its outflow. The model uses these rows to build the routing graph in `hyd_connect`.

Each object type has its own `.con` file but every `.con` file is read by the same routine, `hyd_read_connect`, and is stored in the same array `ob`. The names of the files are listed in `type input_con` in `input_file_module.f90`:

File	Object type passed to `hyd_read_connect`	Default name
`hru.con`	`hru`	`in_con%hru_con`
`hru-lte.con`	`hru_lte`	`in_con%hruez_con`
`rout_unit.con`	`ru`	`in_con%ru_con`
`gwflow.con`	`gwflow`	`in_con%gwflow_con`
`aquifer.con`	`aqu`	`in_con%aqu_con`
`channel.con`	`chan`	`in_con%chan_con`

File	Object type passed to `hyd_read_connect`	Default name
`chandeg.con`	`chandeg`	`in_con%chandeg_con`
`reservoir.con`	`res`	`in_con%res_con`
`recall.con`	`recall`	`in_con%rec_con`
`exco.con`	`exco`	`in_con%exco_con`
`delratio.con`	`dr`	`in_con%delr_con`
`outlet.con`	`outlet`	`in_con%out_con`

A 13th name, `aquifer2d.con` (`in_con%aqu2d_con`), is declared in `type input_con` but is not currently passed to `hyd_read_connect` from `hyd_connect.f90`.

Source

- Reader: src/hyd_read_connect.f90
- Dispatcher: src/hyd_connect.f90 (calls `hyd_read_connect` once per object type)
- Type definition: src/hydrograph_module.f90 (`type object_connectivity`, array `ob`)
- File name pointers: src/input_file_module.f90 (`type input_con`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.
- Line 3 and beyond: one record per object. The loop runs from `nspu1` to `nspu1 + nspu - 1`, where `nspu` is the count for this object type taken from `sp_ob` (set in `object.cnt`). The number of rows in the file must equal `nspu`.

Each record has a fixed leading block of 13 fields followed by `out_tot` connection tuples. The reader makes two list-directed reads of the same line: the first pass reads up to `src_tot`, then if `src_tot > 0` the reader backspaces and re-reads with the connection tuples appended.

Fixed leading block

#	Column (typical header)	Maps to	Type	Description
1	id / NUMB	`ob(i)%num`	int	Object number within this object type. Sequential. Becomes the spatial-object pointer used everywhere else (`hru` column of `hru-data.hru`, channel id in `chandeg.con`, etc).
2	name / NAME	`ob(i)%name`	char(16)	Object name. Free-form.
3	gis_id / GIS_ID	`ob(i)%gis_id`	int*8	GIS identifier from the source watershed delineation.
4	area / AREA_HA	`ob(i)%area_ha`	real (ha)	Drainage area. For HRU, routing unit, and recall objects this value is also copied into `ob(i)%area_ha_calc`. For other objects `area_ha_calc` is initialized to 0 and is filled in by `hyd_connect` from the routing topology.
5	lat / LAT	`ob(i)%lat`	real (deg)	Latitude.
6	lon / LONG	`ob(i)%long`	real (deg)	Longitude.
7	elev / ELEV	`ob(i)%elev`	real (m)	Elevation.

#	Column (typical header)	Maps to	Type	Description
8	hru / aqu / lcha / rtu / RECALL	`ob(i)%props`	int	Pointer to the parameter record for this object. For `hru.con` it points into `hru-data.hru`; for `chandeg.con` into the SWAT-deg channel database; for `aquifer.con` into `aquifer.aqu`; and so on.
9	wst / WST	`ob(i)%wst_c`	char(50)	Weather station name. Resolved by `search(wst_n, db_mx%wst, wst_c, wst)` to the integer `ob(i)%wst` after the record is read.
10	cst / CONSTIT	`ob(i)%constit`	int	Pointer to constituent data (pesticides, pathogens, metals, salts). `0` means no constituent data.
11	ovfl / OVERFLOW	`ob(i)%props2`	int	Overbank-connectivity pointer to a landscape unit. `0` means no overbank connection.
12	rule / RULESET	`ob(i)%ruleset`	char(16)	Name of a decision table in `flo_con.dtl` that controls the outflow fraction. Empty string means no rule.
13	out_tot / OUT_TOT	`ob(i)%src_tot`	int	Number of downstream (source) connections that follow on this line. May be 0.

Per-connection tuple

If `out_tot > 0`, the reader expects `out_tot` groups of four values after column 13, all on the same line:

Offset	Column (typical header)	Maps to	Type	Description
+1	obj_typ / OBTYP_OUT	`ob(i)%obtyp_out(k)`	char(3)	Type of the downstream object. Values used in `hyd_connect`: `hru`, `ru`, `aqu`, `cha`, `sdc` (SWAT-deg channel), `res`, `rec`, `exc`, `dr`, `out`.
+2	obj_id / HTYPNO_OUT	`ob(i)%obtypno_out(k)`	int	Sequential number of the downstream object within its type.
+3	hyd_typ / HTYP_OUT	`ob(i)%htyp_out(k)`	char(3)	Which outflow hydrograph from this object flows to the target. Values include `tot` (total), `rhg` (recharge), `sur` (surface), `lat` (lateral), `til` (tile). The set depends on the source object type; see `type hd_tot` in `hydrograph_module.f90`.
+4	frac / FRAC_OUT	`ob(i)%frac_out(k)`	real	Fraction of that hydrograph routed to the target. The fractions for one source object should sum to 1.0. The reader does not check this.

If `out_tot == 0`, the reader allocates `ob(i)%obtypno_out(1)` and sets it to 0. No connections are recorded.

gwflow side effect

After the connection block is read, if `bsn_cc%gwflow == 1` and any of the connections target an aquifer (`obtyp_out(k) == "aqu"`), the reader decrements `ob(i)%src_tot` by 1. The aquifer connection is left in the arrays but `src_tot` no longer counts it. This is part of the gwflow override that replaces standard aquifer routing with grid-cell routing (see `object.cnt`, gwflow side effect).

Example

`refdata/Osu_1hru/aquifer.con`:

```
aquifer.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       aqu
       1  aqu001                   1      10.00000      35.64206     127.38513     322.26395         1
```

Notes:

- One aquifer object, named `aqu001`, with `area_ha = 10.0` and `out_tot = 1`.
- The single connection sends `tot` (total) outflow from this aquifer to SWAT-deg channel number 1 (`sdc 1 tot 1.00000`), fraction 1.0.

`refdata/Osu_1hru/rout_unit.con` shows a row with `out_tot = 2`:

```
rout_unit.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       rtu
       1  rtu001                   1       10.0000      35.52014     127.32787     113.51276         1
```

The routing unit `rtu001` sends total flow to SWAT-deg channel 1 (fraction 1.0) and recharge flow (`rhg`) to aquifer 1 (fraction 1.0). The two fractions need not sum to 1.0 because they apply to different hydrograph types (`tot` and `rhg`).

`refdata/Ames_sub1/hru.con` is the trivial case with `out_tot = 0` on every row:

```
hru.con: AMES
      id  name                gis_id          area           lat           lon          elev       hru
       1  hru0001                  1         1.005         41.20        -96.63         347.5         1
       2  hru0002                  1         1.005         41.20        -96.63         347.5         2
```

When `out_tot = 0` the row stops at column 13 and no connection tuples follow. Routing for these HRUs is then expected to be supplied through a separate routing unit definition.

Related files

- `object.cnt`. Sets `sp_ob%hru`, `sp_ob%chan`, etc, which determine how many rows `hyd_read_connect` expects in each `.con` file.
- `hru-data.hru`. The `hru` column in `hru.con` points into the rows of this file.
- `hru-lte.hru`, `rout_unit.def`, `aquifer.aqu`, `channel.cha`, `chandeg.cha`, `reservoir.res`, `recall.rec`, `exco.exc`, `delratio.del`. Each `.con` file points into one of these property files via the `props` column.
- `flo_con.dtl`. Decision tables named in the `ruleset` column.
- `weather-sta.cli`. Source of the names in the `wst` column.
- `gwflow.chancells`. When `bsn_cc%gwflow == 1`, the aquifer connection is silently dropped from `src_tot` after reading.

Purpose

`hru-data.hru` is the pointer table for every HRU in the project. Each record names one HRU and lists the parameter sets it draws from: topography, hydrology, soils, land use management, soil and plant initialization, surface storage, snow, and field. The names in this file are looked up in the corresponding parameter files (`topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`). The integer indices resolved by those lookups are stored in `hru_db(i)%dbs` and used throughout the simulation to fetch HRU properties.

If the file is missing or set to `null` in `file.cio`, the reader allocates `hru_db(0:0)` and no HRU data is loaded.

Source

- Reader: src/hru_read.f90
- Type definition: src/hru_module.f90 (`type hru_databases_char`, `type hydrologic_response_unit_db`)
- File name pointer: src/input_file_module.f90 (`in_hru%hru_data`, default `"hru-data.hru"`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.
- Line 3 and beyond: one record per HRU. The reader makes two passes. The first pass scans the file to find the largest `id` and allocates `hru_db(0:imax)`. The second pass reads each record list-directed into `k, hru_db(i)%dbsc`, where `k` is a throwaway read of the id and `i` is the id parsed from the same line.

Record layout: 10 list-directed values per HRU.

#	Column	Maps to	Type	Description
1	id	record index `i`	int	HRU number. Used as the array index into `hru_db`. Does not have to be sequential. The largest `id` in the file sets the array size.
2	name	`hru_db(i)%dbsc%name`	char(40)	HRU name. Free-form identifier.
3	topo	`hru_db(i)%dbsc%topo`	char(40)	Topography record name. Looked up in `topography.hyd`. Missing names are logged to `diagnostics.out` as "not found (topography.hyd)".
4	hydro	`hru_db(i)%dbsc%hyd`	char(40)	Hydrology record name. Looked up in `hydrology.hyd`. Missing names are logged as "not found (hydrograph.hyd)".
5	soil	`hru_db(i)%dbsc%soil`	char(40)	Soil record name. Looked up in `soils.sol`. Missing names are logged as "not found (soils.sol)".
6	lu_mgt	`hru_db(i)%dbsc%land_use_mgt`	char(40)	Land use and management record name. Looked up in `landuse.lum`. Missing names are logged as "not found (landuse.lum)".
7	soil_plant_init	`hru_db(i)%dbsc%soil_plant_init`	char(40)	Soil and plant initialization record name. Looked up in `plant.ini`. Missing names are logged as "not found (plant.ini)". A match also cross-walks the initial soil-test, pesticide, pathogen, heavy-metal, salt, and constituent records named inside the `plant.ini` entry.
8	surf_stor	`hru_db(i)%dbsc%surf_stor`	char(40)	Surface storage (reservoir or wetland) record name. The reader stores the name but does not resolve a pointer for it in `hru_read`. Use `null` if the HRU has no surface storage.
9	snow	`hru_db(i)%dbsc%snow`	char(40)	Snow record name. Looked up in `snow.sno`. The literal `null` is allowed and suppresses the not-found warning.
10	field	`hru_db(i)%dbsc%field`	char(40)	Field record name. Looked up in `field.fld`. The literal `null` is allowed and suppresses the not-found warning.

The reader does not enforce a count line. The number of HRUs comes from the maximum `id` seen during the scan pass, so a gap in the numbering allocates unused slots in `hru_db`.

Example

refdata/Ames_sub1/hru-data.hru:

hru-data.hru:

id	name	topo	hydro	soil	lu_mgt	soil_
1	hru0001	topohru0001	hyd0001	soil_01-h1	cosy_lum	
2	hru0002	topohru0002	hyd0002	soil_02	mntill_corn_lum	
3	hru0003	topohru0003	hyd0003	soil_03-h3	cosy_lum	
4	hru0004	topohru0004	hyd0004	soil_04	cosy_lum	

Notes on the example:

- HRU 2 uses `mntill_corn_lum` while the rest use `cosy_lum`. The land use record drives tillage, plant cover, and management operations.
- `surf_stor` and `field` are `null` for every HRU. None of the HRUs has a pond, wetland, or tile-drained field assigned.
- `snow001` is the same record for every HRU. The snow record is a per-elevation-band parameter set.

Related files

- object.cnt. The `hru` count must be at least the largest `id` in this file.
- hru.con. One row per HRU, in the same id order.
- `topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`. Targets of the name pointers above.
- `file.cio`. The `hru` category line points to this file.

Purpose

This page documents the per-HRU hydrology and topography parameter files: `hydrology.hyd`, `topography.hyd`, and `field.fld`, plus the closely related shade-factor file `shade_factor.shf`. File names for the first three are set in `type input_hydrology` in `src/input_file_module.f90`; the shade file is set in `type shade_factor` in the same module.

Records in `hydrology.hyd`, `topography.hyd`, and `field.fld` are referenced by name from hru-data.hru (columns `hydro`, `topo`, and the field-name slot). Every HRU points at one record from each. Records in `shade_factor.shf` are used by the channel temperature routine in `src/ch_temp.f90`.

Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates a zero-length array and skips on. All four readers skip two header lines, then read one record per parameter set.

Source

- File names: src/input_file_module.f90, `type input_hydrology`, `type shade_factor`
- Readers:
  - src/hydrol_read.f90 (`hydrology.hyd`)
  - src/topo_read.f90 (`topography.hyd`)
  - src/field_read.f90 (`field.fld`)
  - src/shade_factor_read.f90 (`shade_factor.shf`)
- Type definitions: src/hydrology_data_module.f90, src/topography_data_module.f90, src/hydrograph_module.f90

hydrology.hyd

Per-HRU hydrology parameter sets (lateral flow, canopy, evaporation compensation, percolation, enrichment ratios). Read into `hyd_db` (`type hydrology_db` in `src/hydrology_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	lat_ttime	real	days	0.0	exponential of the lateral flow travel time (0..120)
3	lat_sed	real	g/L	0.0	sediment concentration in lateral flow
4	canmx	real	mm H2O	0.0	maximum canopy storage
5	esco	real	none	0.0	soil evaporation compensation factor (0..1)
6	epco	real	none	0.0	plant water uptake compensation factor (0..1)
7	erorgn	real	none	0.0	organic N enrichment ratio (0 = model calculates per event)
8	erorgp	real	none	0.0	organic P enrichment ratio (0 = model calculates per event)
9	cn3_swf	real	none	0.0	soil water at CN3 (0 = field capacity, 0.99 = near saturation)
10	biomix	real	none	0.0	biological mixing efficiency, applied at end of each calendar year
11	perco	real	none	0.0	percolation coefficient, linear adjustment to daily perc (0..1)
12	lat_orgn	real	ppm	0.0	organic N concentration in lateral flow
13	lat_orgp	real	ppm	0.0	organic P concentration in lateral flow
14	pet_co	real	none	1.0	radiation coefficient used in Hargreaves equation
15	latq_co	real	none	0.3	lateral soil flow coefficient, linear adjustment to daily lateral flow (0..1)

Example from `refdata/Osu_1hru/hydrology.hyd`:

```
hydrology.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                 lat_ttime       lat_sed       can_max          esco          epco   orgn_enrich   org
hyd0001                0.00000       0.00000       1.00000       0.95000       0.50000       0.00000
```

Editor column names (`can_max`, `harg_pet`, `bio_mix`) differ from source field names. The reader uses position only.

topography.hyd

Per-HRU topography parameter sets. Read into `topo_db` (`type topography_db` in `src/topography_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	slope	real	m/m	0.02	average slope steepness in HRU
3	slope_len	real	m	50.0	average slope length for erosion
4	lat_len	real	m	50.0	slope length for lateral subsurface flow
5	dis_stream	real	m	100.0	average distance to stream
6	dep_co	real	none	1.0	deposition coefficient

Example from `refdata/Osu_1hru/topography.hyd`:

```
topography.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        slp       slp_len       lat_len      dist_cha         depos
topohru0001             0.01078      10.00000      10.00000     121.00000       0.00000
```

The same type and array are also used for landscape units; `hru-data.hru` and the subbasin objects both reference entries in `topo_db` by name.

field.fld

Per-field geometry used by the wind-erosion routine. Read into `field_db` (`type fields_db` in `src/topography_data_module.f9`

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	length	real	m	500.0	field length for wind erosion
3	wid	real	m	100.0	field width for wind erosion
4	ang	real	deg	30.0	field angle for wind erosion

Example from `refdata/Osu_1hru/field.fld` (first four records shown):

```
field.fld: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       len            wd           ang
fld011                500.00000     100.00000      30.00000
fld012                500.00000     100.00000      30.00000
fld031                500.00000     100.00000      30.00000
fld032                500.00000     100.00000      30.00000
```

The reader uses a `read .. backspace .. read` pattern, but the file format is one record per line, same as the other files on this page.

shade_factor.shf

Time- and landscape-unit-resolved shade factor used by the channel water-temperature routine. Read into `shf_db` (`type shade_factor_data` in `src/hydrograph_module.f90`). Activated per channel by `sf_on = 1` in `temperature.cha`; when `sf_on = 0` the channel uses the calibration-file value instead.

#	Column	Type	Units	Default	Description
1	jday	int	day of year	0	day of year the record applies to
2	lsu	int	none	0	landscape unit number
3	value	real	none	0.0	shade factor value

In `src/ch_temp.f90` the array is scanned for the matching `(jday, lsu)` pair and the `value` is assigned to `ssff`. There is no name column.

No example dataset in `refdata/` ships a `shade_factor.shf` file.

Related files

- `file.cio`. The `hru` line selects `hydrology.hyd`, `topography.hyd`, and `field.fld`; the shade file is listed separately.
- `hru-data.hru`. Every HRU references one entry from each of `hydrology.hyd`, `topography.hyd`, and `field.fld`.
- `temperature.cha`. Controls whether `shade_factor.shf` is consulted (`sf_on` flag).

Purpose

The recall module reads externally provided time series and exposes them as spatial objects in the SWAT+ object graph. A recall object is a point source, an outlet, or a basin inlet whose flow and constituent loads are read from disk instead of simulated. Each recall has its own time step (subdaily, daily, monthly, or annual). The hydrograph delivered by a recall feeds the receiving object through `recall.con`, exactly the same way that an HRU or a channel feeds its downstream link.

Inputs come in two layers:

- A master database file (`recall.rec`) lists every recall object and points to the per-object side files for organic and mineral constituents and, optionally, for pesticides, pathogens, heavy metals, salts, and other constituents.
- Optional parallel database files (`salt_recall.rec`, `cs_recall.rec`) carry the time series of salt-ion and constituent loads when those modules are active.
- Optional spatial-element files (`rec_catunit.def`, `rec_reg.def`, `rec_catunit.ele`) group recall objects into regions used for output aggregation and soft calibration.

Source

- Master reader: `src/recall_read.f90` (`recalldb_read` and `recall_read`)
- Salt reader: `src/recall_read_salt.f90`
- Constituent reader: `src/recall_read_cs.f90`
- Element reader: `src/rec_read_elements.f90`
- Type definitions: `src/recall_module.f90` (`type recall_databases`, `type constituent_file_data`), `src/hydrograph_module.f90` (`type recall_hydrograph_inputs`), `src/constituent_mass_module.f90` (`type recall_salt_inputs`, `type recall_cs_inputs`, `type recall_pesticide_inputs`).

recall.rec (master database)

Lists every recall object and names the side files that hold its time-series data.

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per recall object. Two passes: the first finds the maximum object number, the second reads the records.

Each record is read into `recall_db(i)` (`type recall_databases`):

#	Field	Type	Description
1	id	int	Sequential id. Used as the array index.
2	name	char(13)	Recall object name. Referenced by `recall.con` and by the `obj_typ = rec` slots in other connectivity files.

#	Field	Type	Description
3	org_min	record	Organic and mineral side file. Three subfields read together: `name` (filename), `units` (`mass` or `conc`), `tstep` (`sub`, `day`, `mo`, `yr`).
4	pest	record	Pesticide side file (same three subfields).
5	path	record	Pathogen side file.
6	hmet	record	Heavy-metal side file.
7	salt	record	Salt side file.
8	constit	record	Constituent (other) side file.

`type constituent_file_data` has three character fields read in order: `name(25)`, `units(13)`, `tstep(13)`. List-directed reads consume three tokens per record slot. After reading each row, the master reader calls `recall_read(i)` which opens the file in `recall_db(i)%org_min%name` and reads the time series.

Per-object time series

`recall_read` opens the organic-mineral side file and dispatches on `tstep`:

`tstep`	Storage	Records per year
`sub`	`recall(irec)%hyd_flo(step*366, nbyr)` plus `recall(irec)%hd(366, nbyr)`	Subdaily. Flow is converted from m^3/s to m^3 per subdaily step.
`day`	`recall(irec)%hd(366, nbyr)`	One row per Julian day.
`mo`	`recall(irec)%hd(12, nbyr)`	One row per month.
`yr`	`recall(irec)%hd(1, nbyr)`	One row per year.

Side-file format (`day`, `mo`, `yr`):

- Line 1: title.
- Line 2: `nbyr` (number of years).
- Line 3: header (skipped).
- Lines 4+: per time step, `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then a 22-field `hyd_output` record (flow plus organic, mineral, and physical constituents).

Each `hyd_output` row supplies, in order: `flo`, `sed`, `orgn`, `sedp`, `no3`, `solp`, `chla`, `nh3`, `no2`, `cbod`, `dox`, `san`, `sil`, `cla`, `sag`, `lag`, `grv`, `temp`. Flow is in m^3/s. Sediment is in metric tons. Nutrients and other constituents are in kg. See `type hyd_output` for the full column list.

The reader fast-forwards the side file to the row whose `iyr` matches `time%yrc` (simulation start year). Rows before the simulation start are skipped; rows after `time%yrc_end` end the read. Each row sets `recall(irec)%start_yr` to the first matching year and `recall(irec)%end_yr` to the last row read.

pest.com (recall pesticide side database)

When `pest.com` exists in the working directory, `recall_read` reads it as a parallel pesticide database. It is structured like `recall.rec`: a master file naming per-pesticide files. Each per-pesticide file has its own `nbyr` and time step (1=day, 2=mo, 3=yr) and fills `rec_pest(i)%hd_pest(istep, nbyr)`. The exact column count of the per-pesticide payload depends on `db_mx%pestcom` (the number of pesticides in the constituent database).

salt_recall.rec (salt side database)

Mirror of `recall.rec` for salt loads, read when the salt module is active.

- Line 1: title.
- Line 2: header.
- Lines 3+: one record per recall object. Fields: `id`, `name`, `typ` (1=day, 2=mo, 3=yr, 4=exco crosswalk), `filename`.

For `typ` in `1..3`, the side file is opened with:

- Line 1: title. If the literal token is `Incoming` the source is flagged as originating from outside the watershed (`pts_type = 2`); otherwise `pts_type = 1` (within the watershed).
- Line 2: `nbyr`.
- Line 3: header.
- Lines 4+: `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then `cs_db%num_salts` real values (one per salt ion). The order of ions follows `constituents.cs`.

Each side file fills `rec_salt(i)%hd_salt(istep, nbyr)%salt(1..num_salts)`. For `typ = 4`, the salt record is crosswalked to an `exco` entry; no side file is opened.

cs_recall.rec (constituent side database)

Same structure as `salt_recall.rec`, but for the other-constituent module. Fields per record: `id`, `name`, `typ`, `filename`. Side files have the same five-line header pattern; payload rows carry `cs_db%num_cs` real values that fill `rec_cs(i)%hd_cs(istep, nbyr)%cs(1..num_cs)`. The `Incoming` flag in the title is honoured identically.

Element and region files

Three files in `file.cio`'s `regions` slot group recall objects for output aggregation and soft calibration. Names default to `rec_catunit.def`, `rec_reg.def`, and `rec_catunit.ele` (set by `in_regs%def_psc`, `in_regs%def_psc_reg`, and `in_regs%ele_psc`). All three are optional.

rec_catunit.def (output regions)

Defines the regions used to print recall output. Format:

- Line 1: title.
- Line 2: number of regions, `mreg`.
- Line 3: header.
- Lines 4+: per region: `id`, `name`, `area_ha`, `nspu`, then `nspu` element ids.

When `nspu > 0`, the listed element ids are passed to `define_unit_elements` and stored in `pcu_out(i)%num`. When `nspu = 0`, the region defaults to every recall in the basin (`sp_ob%recall` elements).

rec_reg.def (calibration regions)

Same format as `rec_catunit.def`. Populates `pcu_reg(i)` instead of `pcu_out(i)`. Used for soft-calibration grouping and per-type output.

rec_catunit.ele (element table)

Lists every element that can appear in `rec_catunit.def` or `rec_reg.def`. Format:

- Line 1: title.
- Line 2: header.
- Lines 3+: per element: `id`, `name`, `obtyp`, `obtypno`, `bsn_frac`, `ru_frac`, `reg_frac`.

`obtyp` is the object-type tag (typically `rec`) and `obtypno` is the recall object number. The three fractions are the share of the element in the basin, in its routing unit, and in its region.

Example

`refdata/Osu_1hru/recall.rec`:

```
recall.rec
NUM     NAME    TYPE     FNAME    OBJ_TYP     OBJ_NAME       flo     sed    orgn    sedp     no3    solp
   1    PSTYR    3      RECYR.        hru         hru1   12120.5       0       0       0       0       0
```

```
  1      1          1     2011         hru          hru2     9000.7        0        0        0        0        0
```

This dataset stores one recall object (`PSTYR`) whose annual hydrograph lives in a side file. The trailing 22-column row is the actual `hyd_output` payload that the side file would contain. In a clean layout, the side file (`RECYR.` or similar) is its own file: a title line, a year count, a header, then one or more annual rows.

Related files

- `recall.con`. Spatial-connectivity entries for each recall object. Links the recall to its receiving spatial object.
- `file.cio`. Names the master `recall.rec` (`in_rec%recall_rec`) and the element files (`in_regs%def_psc`, `in_regs%def_psc_reg`, `in_regs%ele_psc`).
- `salt module`, `cs module`. Document the salt and other-constituent modules whose recall side databases parallel `recall.rec`.